PID-RS TECHNICAL REPORT SERIES

Formal Tool Adoption Audit

Verus, Kani, Aeneas, MPFR through Rug, and Rocq Interval

pid-rs project analysis

25 July 2026

Abstract

This document records the adoption and implementation status of five assurance layers proposed for `pid-rs`. It separates upstream facts, local observations, implemented repository evidence, and future work. Commit `e13c5748ff38daf4c4a88c662434986e3e92b2c2` introduced a standalone, source-only Rug/MPFR reference certifier for all 24 averaged categorical SxPID2 cumulative and atom coordinates. The current repository source also implements an independent Python verifier that reconstructs the event semantics and exact log-linear coordinates from the count table, then proves rational-log enclosures contained in the certificate's dyadic intervals. Report schema v2 adds a separately bounded exact-rational-product comparison that can decide exact zero or strict sign even when the interval-local decision remains unresolved. Seven Lean theorems kernel-check the generic log-product/sign reduction and one retained five-factor identity; they do not formalize SxPID event extraction or refine either executable. These are distinct evidence roles. They are not formal verification of Python or either program, and none refines or certifies `pid-core` binary64 output. The remaining order is bounded Kani harnesses, followed by a Verus or Creusot proof pilot for a small pure categorical kernel, an optional Rocq Interval lane, and a deferred Aeneas translation experiment. The repository now contains the standalone dependency and lockfile, the independent verifier and qualification harnesses, CI gates, method provenance, tests, and a license boundary. No end-to-end verification claim follows.

Contents

1 Record status and purpose

Record status: Adoption and implementation-status record, amended on 25 July 2026. Commit `e13c5748ff38daf4c4a88c662434986e3e92b2c2` is the historical certifier baseline. An archived evidence bundle must additionally bind the repository revision and artifact digests for the independent verifier source that it executes.

Evidence status: No evaluated tool has verified `pid-rs` end to end. The repository does contain one implemented exact-count assurance family with complementary, noninterchangeable parts. Its source-only Rug/MPFR producer emits outward dyadic enclosures of all 24 averaged categorical SxPID2 expressions. Its independent verifier reconstructs the event masses, exact rational coefficients, fixed lattice transform, and all coordinate expressions without accepting the producer's semantic conclusions, and uses an integer/Fraction rational-log argument to prove containment in the emitted intervals. Both implementations separately reconstruct the bounded denominator-cleared rational product used for exact zero/strict-sign decisions. A Lean module kernel-checks only the abstract log-product/sign algebra and one concrete product-one witness. This is not a formal proof of either implementation, the Python runtime, native libraries, compiler, `pid-core`, sampling assumptions, or application validity. Kani, Verus, Rocq Interval, and Aeneas remain unevaluated pilots or experiments in this record.

The audit covers:

1. Verus;
2. Kani;
3. Aeneas with Charon;
4. MPFR through Rug; and
5. Rocq with the Interval package.

The cut-off date is 24 July 2026. Upstream state can change after that date. A pilot must repeat the upstream release check before installation.

The decision is:

1. Retain the implemented Rug/MPFR exact-count SxPID2 certifier as a standalone, source-only reference producer together with the independently implemented exact-integer and rational-log verifier and the bounded exact-product zero/sign extension. Retain the generic Lean log-product/sign theorem as a separate formal-algebra artifact. Do not merge either executable into the stable binary64 API or distribute a compiled artifact without a separate license and build-evidence review.
2. Pilot Kani next on bounded categorical integer, event, lattice, overflow, and rejection obligations.
3. After bounded bit-precise checks, pilot deductive contracts on the same small, pure categorical kernel with Verus or Creusot. This record fully audits Verus only; Creusot requires its own pin, license, command, and TCB record before it contributes evidence.
4. Evaluate Rocq Interval only after the independent verifier's expression semantics, input binding, and containment contract are frozen as a versioned specification.
5. Defer Aeneas production adoption. Permit one isolated translation experiment.

No one tool replaces Lean. The tools cover different failure classes. Evidence from one layer does not imply evidence from another layer.

2 Scope, exclusions, and terms

This record covers tool fit, versions, licenses, toolchains, installation pins, reproducible commands, trust boundaries, and permitted claim language.

This record and the implemented first lane do not:

- change a public `pid-core` API or the existing estimator's numerical result;
- prove Rust-to-specification or certifier-to-`pid-core` refinement;
- deductively prove the certifier, the independent Python verifier, or their runtimes;
- prove compiler, FFI, GMP, MPFR, Rug, or native-archive correctness;
- prove statistical calibration, population assumptions, or a theorem about continuous PID;
- certify pointwise, fitted-quantized, higher-source, or $I_{\min}$ PID;
- approve a linked executable for distribution; or
- claim that a local build is a release or binary attestation.

License identifiers come from upstream metadata. They are not legal advice. A distribution review is mandatory before code that links to an LGPL or CeCILL-C component is released.

Upstream fact. A fact in an official project repository, release page, manual, or package record.

Local fact. An observation from the workstation on 24 July 2026.

Trust boundary. Code, data, assumptions, or tools that a proof result does not verify. This document also uses the common term trusted computing base, or TCB.

Pilot. A bounded evaluation that has not yet become repository evidence. The Rug/MPFR producer and independent Python verifier are no longer merely pilots; the other four fully audited tool lanes remain pilots or experiments.

Certified reference tool. The standalone source-only tool under `audit/tools/certified-sxpid`. It returns outward intervals for the exact-real expressions that its own reviewed extractor encodes. It is not a `pid-core` backend and does not certify an existing binary64 result.

Independent certificate verifier. The standard-library Python program `audit/tools/certified-sxpid/scripts/verify_certificate.py`. It treats the count table and certificate as untrusted inputs, independently reconstructs the SxPID2 expressions, and proves that its rational-log enclosures are contained in the reported dyadic intervals. "Independent" means that it does not call or import the Rust extractor, Rug, MPFR, or GMP. It does not mean independent authorship or custody, and it does not mean that the Python program or interpreter has been formally verified.

Exact-product decision. A separately bounded comparison of the positive rational product obtained after clearing the empirical denominator. A successful comparison proves exact zero or strict sign for the frozen empirical coordinate, conditional on the implementation or formal route that establishes the product identity. It does not enclose a nonzero magnitude and does not rewrite the interval-local decision.

3 Current repository and local state

3.1 Repository pins

Item	Repository pin	Source
Minimum supported Rust version	1.89	`Cargo.toml`
Lean	4.32.0	`audit/formal/lean/lean-toolchain`
mathlib	`v4.32.0`	`audit/formal/lean/lakefile.toml`
Existing finite-convergence Lean proof-surface gate	225 ordered source declarations, 177 theorem axiom-basis checks, and 10 digest-pinned semantic examples	`scripts/check-lean-finite-convergence.py`
Exact log-product Lean gate	7 kernel-checked theorems; generic algebra plus one retained five-factor product-one identity	`scripts/check-lean-exact-log-product.py`
Z3 for the existing algebra checker	4.16.0, 64 bit	`scripts/check-z3-pid2-algebra.py`
Standalone exact-count certifier	Rust 1.89, `publish=false`	`audit/tools/certified-sxpid/Cargo.toml`
Rug for the standalone certifier	1.30.0; requested features `float`, `rational`, `std`	`audit/tools/certified-sxpid/Cargo.toml`
Transitive native-sys crate	`gmp-mpfr-sys` 1.7.1	`audit/tools/certified-sxpid/Cargo.lock`
Certificate schemas	Count table v1, exact-log-linear v1, report v2, independent verification v2, resource policy v2, and build context v1	`audit/tools/certified-sxpid/src/resource.rs` and `audit/tools/certified-sxpid/src/report.rs`
Qualification routes	Stable Rust CI, Rust 1.89 CI, 41 Rust tests, Clippy, rustdoc, static policy, 34 static-policy mutations, independent-verifier challenges, exact-product challenges, Lean, and `cargo-deny`	`.github/workflows/ci.yml` and `justfile`

These pins do not apply automatically to a new verifier. Each verifier can have a different compiler and solver pin.

3.2 Local observations

The following commands were run from the repository worktree:

```
rustc --version
cargo --version
(cd audit/formal/lean && lake env lean --version)
z3 --version
pkg-config --modversion mpfr
pkg-config --modversion gmp
command -v <tool>
```

Local item	Observation
`rustc`	`rustc1.96.0(ac68faa202026-05-25)`
`cargo`	`cargo1.96.0(30a34c6822026-05-25)`
Lean	4.32.0, `arm64-apple-darwin24.6.0`, commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`
Z3	4.16.0, 64 bit
MPFR found by `pkg-config`	4.2.2
GMP found by `pkg-config`	6.3.0
FLINT found by `pkg-config`	Not found
Rug/native-sys lock state	Absent from the root workspace lockfile; Rug 1.30.0 and transitive `gmp-mpfr-sys` 1.7.1 are present in the standalone certifier lockfile

The commands `verus`, `cargo-verus`, `kani`, `cargo-kani`, `charon`, `aeneas`, `opam`, `rocq`, `rocqchk`, `coqc`, `coqchk`, and `nix` were not found.

This local list is an environment observation only. It is not an installation attestation. System `pkg-config` discovery is not used as evidence for the certifier. The committed qualification route instead checks the standalone locked Cargo graph offline and requires the transitive `gmp-mpfr-sys` node to resolve exactly feature `mpfr`; it also requires direct command-line injection of `gmp-mpfr-sys/use-system-libs` to fail. The certificate still does not bind effective dependency-feature resolution, compiled native version constants, native archives, executable bytes, compiler wrappers, effective flags, linker/compiler identity, or cache contents.

4 Decision matrix

Layer	Decision	First permitted scope	Main reason	Main blocker
Rug/MPFR producer	Implemented source-only reference	Canonical exact count tables; all 24 averaged categorical SxPID2 cumulative and atom coordinates	Exact rational expression construction, explicit directed log/scaling/accumulation, dyadic endpoints, adaptive precision, and strict failure envelopes	Extractor and wrapper are not deductively verified; native C/FFI/compiler/build evidence remains trusted; no `pid-core` refinement or compiled-artifact distribution

Layer	Decision	First permitted scope	Main reason	Main blocker
Independent verifier	Implemented executable cross-check	Independent reconstruction and containment checks for accepted exact-count SxPID2 certificates	Exact integers and Fractions, fixed SxPID2 semantics, and a separate rational-log enclosure argument without Rug/MPFR conclusions	Python, its integer/Fraction semantics, verifier source, and the enclosure argument remain trusted; no formal proof or `pid-core` refinement
Exact-product extension	Implemented bounded zero/sign evidence	Every averaged SxPID2 coordinate whose preflight status is `compared`	Exact denominator clearing and positive-rational numerator/denominator comparison; the generic sign theorem is separately kernel-checked in Lean	Producer/verifier refinement is not proved; resource abstentions provide no zero/sign decision; the Lean theorem does not bind SxPID events, lattice code, or executable bytes
Kani	Next pilot	Bounded integer, mask, event, lattice, overflow, and rejection harnesses	Exhaustive bit-precise checks inside declared bounds	Bounds do not generalize; concurrency and transcendental accuracy are out of scope
Verus or Creusot	Pilot after Kani	Small pure categorical kernel with explicit contracts	Closest fit for unbounded functional contracts over Rust-like code	The selected verifier, toolchain compatibility, unsupported code, and assumptions require a separate recorded evaluation; the Verus route has the detailed record below
Rocq Interval	Later optional pilot*	Independent checks of fixed emitted interval inequalities	Small kernel checker and analytic interval tactics give a second proof route	Separate ecosystem; the certificate generator and runtime binding remain trusted
Aeneas and Charon	Defer; one experiment	Translation of the same small pure kernel to Lean	Can connect accepted safe Rust to a theorem-prover model	No stable release, Lean 4.31 mismatch, supported-subset limits, and handwritten external models

* The repository now has versioned count-table, exact-expression, build-context, resource-

policy, report, and independent-verification schemas, plus an independent executable reconstruction of their SxPID2 semantics. This satisfies an executable semantic-cross-check precondition for a Rocq pilot. It does not turn the schema into a kernel-checked formal semantics or verify the Python reconstruction.

The implemented first lane supplies a numerical reference certificate for its own encoded expressions. The independent verifier now supplies a separate executable reconstruction and analytic containment route. The bounded exact-product extension and its Lean algebra theorem add exact zero/sign evidence, not magnitude enclosure or executable refinement. The next bridge is bounded compiled-behavior evidence: Kani should target the same integer/event/lattice semantics before an unbounded Verus or Creusot contract pilot. Rocq must not be treated as independent merely because it checks a theorem generated from the Rust producer or copied from the Python verifier. Aeneas must not become a release dependency during this evaluation.

5 Verus adoption record

5.1 Upstream pin and license

The selected non-prerelease cut is `release/0.2026.07.18.3a4d30b`. GitHub records its publication on 20 July 2026.

The release pins:

- Rust 1.96.0;
- `vstd` version `0.0.0-2026-07-12-0122`.

The Z3 fetch helper in the tagged source pins Z3 4.12.5.

The license is MIT. At the audit date, the release page had assets for arm64 macOS, x86-64 macOS, x86-64 Linux, and x86-64 Windows. This list is not a general platform support theorem.

Primary pin sources are the release toolchain, `vstd` package metadata, Z3 fetch helper, and MIT license.

5.2 Pin and installation rule

Do not use a rolling prerelease tag. Do not mix `vstd`, the Verus executable, and solver files from different releases.

For a pilot:

1. Select the platform asset from the immutable release page.
2. Compute and record its SHA-256.
3. Extract it into a version-specific directory.
4. Use `cargo-verus` from that release.
5. Use the solver in the release bundle, or the exact upstream solver pin.
6. Record all tool versions, asset digest, host, and target.

The Cargo metadata and commands are:

```
[package.metadata.verus]
verify = true

cargo verus verify --workspace --locked
cargo verus build --release --locked
```

Verus also supports `--offline`. A release evidence job must use a lockfile and an archived dependency set before it claims offline reproducibility. A focused verification run is a development aid. It is not a substitute for the full verification command.

5.3 First obligations

The pilot can cover:

- multiset preservation by sorting and run-length encoding;
- histogram count totals;
- equality of event counts and event predicates;
- denominator positivity for supported keyed rows;
- canonical source-mask and antichain order;
- Möbius reconstruction; and
- specialized and general categorical-kernel equivalence.

The pilot must not start with logarithms, binary64 error, the complete report API, parallel code, cancellation plumbing, allocators, Python bindings, or continuous estimators.

5.4 Trust boundary, blockers, and negative controls

Verus does not support all Rust code and libraries. The Verus verifier, Rust compiler, LLVM, solver, and platform remain in the trust boundary.

The features `assume`, `external_body`, `external_fn_specification`, and `external` add assumptions or trusted specifications. The pilot must create an assumption inventory. A result with an unreviewed assumption is not accepted.

Verus uses Rust 1.96. The repository supports Rust 1.89. A successful Verus run does not prove that the erased program builds at Rust 1.89. The normal Rust 1.89 gates remain separate.

Required negative controls must:

- mutate an event union to an intersection;
- mutate a source mask;
- remove a Möbius predecessor;
- add an unapproved assumption; and
- run the erased source under Rust 1.89.

The first three mutations must fail a contract. The assumption mutation must fail policy. The MSRV run must detect incompatible erased source.

5.5 Permitted claim

> Verus proved the stated contracts for the opted-in functions under the recorded assumptions and pinned Verus, solver, and toolchain. It did not verify rustc or LLVM, unchecked specifications or external bodies, callers outside those contracts, or floating-point transcendental correctness.

Do not state that Verus verified `pid-rs`, the Rust binary, numerical correctness, or statistical correctness.

6 Kani adoption record

6.1 Upstream pin and license

The selected stable release is `kani-0.67.0`, published on 16 January 2026. It pins Rust `nightly-2025-11-21` and CBMC 6.8.0. Kani is dual licensed under MIT or Apache-2.0.

Primary pin sources are:

- Rust toolchain;
- dependency pins;
- MIT license; and
- Apache-2.0 license.

The official easy-install matrix lists x86-64 Linux with GNU libc, x86-64 macOS, and arm64 macOS. The installation guide states Rust 1.58 or newer through `rustup`.

6.2 Pin and installation rule

Use a version-specific `KANI_HOME`:

```
export KANI_HOME=/absolute/tool-cache/kani/0.67.0
cargo install --locked kani-verifier --version 0.67.0
cargo kani setup
cargo kani --version
```

Archive or hash the resulting tool directory. Record the package checksum, all versions, host, target, harness, input bounds, unwind bound, and unwinding assertion result.

```
cargo kani --harness <HARNESS_NAME> --unwind <BOUND>
```

6.3 First obligations

Use Kani for bounded checks of no panic, no out-of-bounds index, no checked integer overflow, resource-estimate arithmetic, mask and lattice indexing, malformed-shape rejection, cancellation paths, specialized and general kernel equivalence, and exact reconstruction identities.

Each harness must declare all shape and count bounds. It must retain an unwinding assertion.

6.4 Trust boundary, blockers, and negative controls

A result for bound N does not imply a result for $N+1$. Kani does not model supported concurrent execution as a concurrent proof. Its floating-point support includes partial or overapproximated intrinsics. It is not a precision proof route for logarithms.

The Kani compiler, CBMC, solver backend, harness assumptions, and environment models remain in the trust boundary.

Required negative controls must:

- reduce an unwind bound and trigger its assertion;
- introduce an index error and obtain a counterexample;
- remove an overflow check and obtain a counterexample;
- change a specialized mask and obtain an equivalence counterexample; and
- label one case outside the proved bound as not covered.

6.5 Permitted claim

> Kani exhaustively checked this harness within the declared input and unwind bounds using Kani 0.67.0 and CBMC 6.8.0. It does not establish correctness outside those bounds, concurrent execution semantics, or numerical precision of transcendental functions.

Do not state that Kani proved the algorithm for all inputs, verified parallel execution, or proved the accuracy of `ln` or digamma.

7 Aeneas and Charon adoption record

7.1 Upstream pin and license

Aeneas had no stable release at the audit date. Its release page contained nightly prereleases. This record therefore uses:

- Aeneas commit `6e167c9b63a4dafd66d0e0edd9d94669f957ff7b`;
- Charon commit `527ea8e3b5dcb52edd6aef0f7bc34cc09c11dd59`;
- Charon Rust nightly `nightly-2026-06-01`;
- Lean 4.31.0;
- mathlib `v4.31.0`; and
- OCaml switch example 5.3.0.

> Aeneas and Charon are licensed under Apache-2.0.
> Primary pin sources are:

- repository;
- release page;
- Charon pin;
- Charon toolchain;
- Charon Apache-2.0 license;
- Lean toolchain;
- mathlib pin; and
- license.

7.2 Reproducible experiment rule

Use a detached full commit. Do not use a moving nightly name as evidence identity.

```
git clone https://github.com/AeneasVerif/aeneas.git
git -C aeneas checkout --detach 6e167c9b63a4dafd66d0e0edd9d94669f957ff7b
cd aeneas
opam switch create 5.3.0
eval "$(opam env)"
make setup-charon
make
```

`makesetup-charon` must resolve the exact Charon commit. Export the complete OPAM switch. Retain `flake.lock` if Nix is used.

```
charon cargo --preset=aeneas
./bin/aeneas -backend lean [OPTIONS] <LLBC_FILE>
```

Retain both commits, Charon nightly, OPAM export, Lean and mathlib pins, translation flags, source digest, LLBC digest, generated Lean digest, and Lean checker output.

7.3 Experiment scope and blockers

Translate only the same small pure kernel selected for Verus. Compare the generated statement with the handwritten Lean specification. The experiment must identify rejected constructs, handwritten external models, theorem correspondence, and toolchain compatibility.

Aeneas supports a subset of safe Rust. Unsafe code and concurrency are out of scope. Some loop control patterns and library operations are unsupported. Handwritten models become trusted inputs.

The backend pins Lean and mathlib 4.31.0. The repository pins 4.32.0. This mismatch blocks direct integration.

Charon translation, Aeneas translation, handwritten models, theorem correspondence, Lean, and the compiler path remain in the trust boundary.

Required negative controls must submit an unsupported construct, mutate an event predicate, remove a handwritten model, alter the Charon checkout, and compare Lean 4.31.0 with 4.32.0. Rejection must be explicit. A compatibility experiment must not be reported as toolchain equivalence.

7.4 Permitted claim

> Aeneas translated the accepted safe-Rust subset at the pinned Aeneas and Charon commits, and Lean checked the generated model's stated theorem. This does not establish translation support for all Rust, correctness of handwritten external models, unsafe or concurrent code, or binary equivalence unless those links are separately proved.

Do not state that Aeneas verified the Rust implementation, that generated Lean proves the compiled binary, or that the translation covers unsupported library code.

8 Rug and MPFR adoption record

8.1 Upstream pin and license

The standalone tool locks:

- Rug 1.30.0;
- transitive `gmp-mpfr-sys` 1.7.1; and
- manifest-requested Rug features `float`, `rational`, and `std`.

The selected `gmp-mpfr-sys` release supplies GMP 6.3.0 and MPFR 4.2.2 sources; MPC is not selected by the committed feature graph. The runtime certificate reports the locked crate versions, not compiled native version constants or native archive identities. Consequently the compiled executable must not be claimed to bind GMP 6.3.0 or MPFR 4.2.2 without an external build evidence envelope.

Rug requires Rust 1.85 or newer. `gmp-mpfr-sys` requires Rust 1.71 or newer. Both fit the repository Rust 1.89 minimum.

Rug, `gmp-mpfr-sys`, and MPFR use LGPL-3.0-or-later licensing in the cited package records. GMP 6.3.0 is available under LGPLv3 or GPLv2. A distribution review is mandatory.

Primary sources are:

- MPFR current release;
- MPFR manual;
- Rug 1.30.0 metadata;
- Rug rounding modes;
- `gmp-mpfr-sys` metadata; and
- GMP copying conditions.

MPFR states that most functions are correctly rounded as if computed at infinite precision and then rounded in the requested direction. `mpfr_log` computes the natural logarithm. `MPFR_RNDD` rounds toward negative infinity. `MPFR_RNDU` rounds toward positive infinity. Experimental faithful rounding `MPFR_RNDF` must not be used for a certificate.

8.2 Committed standalone dependency pin and qualification route

The committed source-only tool uses:

```
[dependencies]
rug = { version = "=1.30.0", default-features = false,
        features = ["float", "rational", "std"] }
```

It deliberately has no direct `gmp-mpfr-sys` dependency, which removes the direct dependency-feature injection surface. Its static qualification gate parses the locked graph offline, requires the transitive native-sys node to resolve exactly `["mpfr"]`, rejects `use-system-libs` in the manifest, and requires direct command-line feature injection to fail. The project publishes neither the package nor a compiled certifier artifact. This qualification does not bind the effective build after compiler wrappers, flags, linker/native compiler, or cache selection. Native archive and executable digests remain absent and must be supplied externally when a binary-level claim requires them.

8.3 Implemented numerical obligations and explicit exclusions

For one canonical exact two-source categorical count table, the tool:

1. keeps counts as exact integers and constructs exact rational log coefficients and arguments;
2. reconstructs four informative, four misinformative, and four signed-net cumulative expressions;
3. applies the pinned two-source Möbius matrix at the exact symbolic-expression layer, not to already rounded intervals, and constructs all twelve atom expressions;
4. evaluates every rational conversion, logarithm, signed coefficient multiplication, and accumulation with explicit lower or upper rounding;
5. converts authoritative endpoints to normalized exact dyadics;
6. evaluates at 128, 256, 512, 1024, 2048, and 4096 bits, intersects successive enclosures, and requires every final width to be at most 2^{-160};
7. fails with a typed precision-limit result when the policy is exhausted and treats an empty successive intersection as an internal soundness failure;

8. reports `certified_positive`, `certified_negative`, `unresolved_sign`, or `certified_exact_zero`; exact zero is reserved for a canonical symbolic expression with no remaining terms; and

9. separately clears the empirical denominator and, when the exact-product resource preflight succeeds, compares one exact positive rational product with one to decide exact zero or strict sign without calling a logarithm; and

10. binds raw and semantic inputs, exact terms, expressions, lattice, precision/resource policy, source manifest, lockfile, and the canonical payload by digest.

An interval containing zero is still a certified enclosure and is labelled `unresolved_sign`; it is not converted into an interval-local sign claim. A separate exact-product record with status `compared` may nevertheless certify exact zero or strict sign. A product preflight abstention leaves that decision unavailable and does not invalidate the value enclosure. The interval remains the magnitude-enclosure authority. $I_{\min}$ and minimizer ordering are outside this tool's schema rather than unresolved outputs. Pointwise SxPID, fitted quantization, higher-source SxPID, continuous KSG/$I_{\cap}^{\mathrm{sx}}$/PID, population assumptions, calibration, and `pid-core` binary64 refinement are also excluded.

8.4 Implemented independent reconstruction and containment route

The repository also implements `audit/tools/certified-sxpid/scripts/verify_certificate.py`. This verifier uses only the Python standard library and deliberately does not import or call the Rust certifier, Rug, MPFR, GMP, NumPy, SymPy, or another numerical package. It:

1. parses the count table and certificate as untrusted, bounded, canonical inputs;

2. independently reconstructs the four source-event unions, their target intersections, all informative, misinformative, and signed-net cumulative expressions, the fixed two-source Möbius transform, and all twelve atom expressions;

3. represents counts and coefficients with arbitrary-precision integers and `fractions.Fraction`;

4. independently clears the empirical denominator, repeats the bounded exact-product preflight and rational comparison, and checks the producer's separate exact-product status, decision, witness, and resource trace;

5. for each positive rational x, range-reduces $x = 2^e y$ with $1 \le y < 2$, puts $z = (y-1)/(y+1)$ in $[0, 1/3]$, and encloses
$$\log y = 2 \sum_{k \ge 0} \frac{z^{2k+1}}{2k+1}$$
by outward integer fixed-point rounding and the explicit omitted-tail bound
$$0 \le 2 \sum_{k \ge m} \frac{z^{2k+1}}{2k+1} \le \frac{9 z^{2m+1}}{4(2m+1)};$$
it encloses $\log 2$ by the same series and combines signs outward in $\log x = e \log 2 + \log y$;

6. raises fixed-point precision until the independently derived enclosure is proved to be a subset of the dyadic interval asserted by the certificate; and

7. rejects a semantic, lattice, expression, interval, claim, policy, or bound-artifact mismatch, and rejects failure to prove containment rather than accepting numerical proximity.

The companion qualification program `audit/tools/certified-sxpid/scripts/check-independent-verifier.py` exercises the verifier on an independently generated exhaustive small-table corpus, compares directly reconstructed mutual information identities, and retains fail-closed semantic mutations, malformed and resource-amplifying inputs, cross-artifact binding adversaries, optimizer-mode execution, and hash-seed-invariance checks. These controls are evidence about the exercised implementation. They are not an exhaustive proof of the parser, source binding, arithmetic, or mathematical argument.

8.5 Trust boundaries, qualification evidence, and remaining gaps

For the producer-only route, the SxPID event/expression extractor, source wrapper, static-policy adequacy, Rug, MPFR, GMP, `gmp-mpfr-sys`, native build, Rust compiler, C ABI and FFI, effective dependency features and flags, linker/native compiler, cache contents, native archives, and executable bytes remain trusted or unbound. The standalone crate forbids unsafe Rust in its own source, but that restriction does not verify native transitive code.

The independent verifier does not accept the producer's event, expression, lattice, or interval conclusions without reconstruction and containment. Its own trusted base instead includes the Python interpreter, arbitrary-precision integer and Fraction semantics, JSON/TOML/path and SHA-256 implementations, verifier and qualification source, file-system observations, and the reviewed rational-log derivation and rounding argument. It is executable cross-check evidence, not a proof object checked by a small formal kernel. Input authenticity and scientific meaning remain unbound under both routes.

The Rust suite currently lists 43 tests: 37 library tests, four CLI contract tests, one exhaustive oracle integration test, and one resource-adversary integration test. They cover exact XOR and logarithm identities, interval and exact-product sign boundaries, non-finite and reversed endpoint rejection, event-mass ordering, empty-intersection and precision-exhaustion failures, vector-state and 1000-digit common-count metamorphisms, aggregate product-preflight abstention, and strict parser failures. Process and integration evidence additionally covers 11,856 all-coordinate tolerance-overlap comparisons and 1,482 direct-MI comparisons over 494 independently generated binary empirical tables, literal-pinned fixture and generator bytes, and 34 fail-closed static-policy mutations. The Decimal corpus is bounded numerical agreement, not a rigorous oracle enclosure.

The independent verifier qualification reconstructs 11,856 coordinates, 1,482 direct-MI identities, and 5,928 direct row-scan event-expression identities over the same 494-table domain; it proves 72 live-certificate containments and 975 exact-`Fraction` log enclosures. Its retained controls kill 23 certificate/input semantic mutations, one fixed-point source mutation, and one event-extraction source mutation, and reject four cross-artifact, six structural, and two transport/invocation adversaries. A distinct exact-product self-test kills 13 certificate mutations, six source-semantic/arithmetic mutations, and four structural adversaries. It also passes two patched-power sentinel controls showing that local and aggregate preflight rejection occurs with zero calls to the auxiliary power primitive; these are not a formal runtime cost theorem. The bounded exact-product qualification compares all 11,856 coordinates; a second exhaustion checks 308,856 coordinates from all 12,869 nonzero binary tables through total count eight and retains all 16 nonempty product-one cases at total eight. Seven Lean theorems kernel-check the generic log-product/sign reduction and one retained five-term identity under their recorded axiom basis. None of these finite corpora or mutation suites is complete program verification. The independent verifier materially reduces common-mode dependence on the Rust extractor and Rug/MPFR arithmetic conclusions, but it does not deductively prove either implementation or eliminate the verifier TCB described above. The Lean theorem does not supply the missing executable or SxPID-semantic bridge.

8.6 Permitted claim

> For a canonical exact two-source empirical count table and certificate accepted by the recorded independent verifier, the verifier reconstructed the declared SxPID2 event semantics, four-node lattice, and all 24 exact log-linear coordinates, and proved that its rational-log enclosure for each coordinate is contained in the certificate's dyadic interval. This claim is conditional on the recorded verifier source, Python runtime and arbitrary-precision arithmetic semantics, bound schemas and artifacts, file-system observations, and reviewed enclosure argument. It is not a formal verification result. `pid-core` binary64 refinement, input authenticity, population and sampling assumptions, calibration, and downstream application validity are absent and are not claimed.
>
> For a coordinate whose independently validated exact-product record has status `compared`, the same route additionally proves exact zero, positivity, or negativity by exact rational-product comparison after denominator clearing. That decision is separate from the dyadic enclosure and does not change an interval-local `unresolved_sign` result. The seven Lean theorems establish the generic real-log/product/sign implication and one retained rational identity only; they do not prove that either executable constructed the intended SxPID coordinate.

Do not state that MPFR or Python formally verified the result, that the current `f64` output is certified, that the producer or verifier proves itself, or that an interval proves sampling assumptions or estimator calibration.

9 Rocq Interval adoption record

9.1 Upstream pin and license

The selected stable pins are Rocq 9.2.0, released on 27 March 2026, and `coq-interval` 4.11.4, published on 23 February 2026. Rocq 9.3 release-candidate builds are prereleases and are not selected.

Rocq core is LGPL-2.1-only. `coq-interval` is CeCILL-C.

The package archive SHA-512 for `coq-interval` 4.11.4 is:

```
5b2ccff32c3d6caa9002455dc472d6c4c8f70337581d038ca432dd92ed90b262
b203960a2a62896c12992be2ff2436bf7e6a8ffd3aec625859f47088d262ef00
```

The two lines are one continuous digest.

Primary sources are:

- Rocq site;
- Rocq 9.2.0 release;
- core package metadata;
- Interval package record;
- Interval versions;
- Interval documentation;
- exact OPAM metadata;
- Rocq OPAM instructions; and
- compile and check commands.

The exact OPAM metadata permits the modern split `coq-core` and `coq-stdlib` route. It also requires Flocq, MathComp, Coquelicot, bignums, OCaml, and a compiler.

9.2 Pin and installation rule

```
opam switch create 4.14.2
eval "$(opam env)"
opam repo add rocq-released https://rocq-prover.org/opam/released
opam install rocq-prover=9.2.0 rocq-core=9.2.0 coq-interval=4.11.4
opam pin add rocq-core 9.2.0
opam pin add coq-interval 4.11.4
opam switch export --full rocq-9.2.0-interval-4.11.4.export
```

Retain exact OPAM version, repository commit, full switch export, package hashes, certificate source digest, compiled object digest, and checker output with assumptions.

```
rocq compile -q Certificate.v
rocq check -o Certificate.vo
```

The `-o` output exposes the logical context and assumptions. The evidence gate must reject `Admitted`, `admit`, unexpected axioms, and an incomplete check.

9.3 First obligations

The repository now emits versioned count-table, exact-log-linear, and report schemas. Its independent Python verifier already reconstructs the event masses, exact rational coefficients, fixed lattice order, all 24 coordinate identities, and bound artifact identities without trusting the Rug evaluator's conclusions. That executable verifier supplies a concrete semantic specification and counterexample corpus for a Rocq pilot, but it is not a kernel-checked theorem. Before a Rocq lane is credited as an additional independent route, its statement generator and proof must reconstruct or bind those obligations without merely copying conclusions from either the Rust producer or the Python verifier.

Candidate theorems can state:

- an independently reconstructed rational logarithmic expression lies inside emitted dyadic endpoints;
- strict endpoint inequalities justify a positive or negative sign decision;
- an interval-local exact-zero status has the declared empty-expression witness, or a separately admitted exact-product decision has the declared denominator-cleared product-one witness;
- the pinned zeta and Möbius matrices reconstruct cumulative and atom coordinates; and
- the theorem binds the exact count table, expression schema, coordinate order, and payload digests.

The theorem statement must bind to the exact input identity. A theorem about the wrong count table can still be a valid theorem. Agreement with the Python verifier is a useful differential check, not a substitute for a Rocq proof or an independent review of the generated statement.

$I_{\min}$ ordering requires a separate future schema and cannot be inferred from the present SxPID2 certificate.

9.4 Trust boundary, blockers, and negative controls

Rocq's kernel checks the proof object. It does not prove that the generator encoded the intended formula or data, that a file digest was bound to a runtime operation, that `pid-rs` used the result, or that the statement has statistical meaning.

The generator, statement schema, input binding, serialization code, OCaml runtime, Rocq kernel, and platform remain in the trust boundary.

Required negative controls must mutate an endpoint, mutate the bound input digest, insert an admission, add an unexpected axiom, and present a valid proof for a different count table. Each control must fail the applicable proof or identity gate.

9.5 Permitted claim

> Rocq's kernel independently checked the emitted inequality theorem and exposed its remaining assumptions. It does not prove that the generator encoded the intended data and expression or that the `pid-rs` runtime used that certificate.

Do not state that Rocq verified the Rust implementation, proved input provenance, or proved application validity.

10 Cross-layer evidence rules

10.1 A result stays inside its layer

Evidence	What it can support	What it cannot support by itself
Existing Lean theorem	Exact theorem in the declared Lean model	Rust implementation, binary64 output, or data assumptions
Exact log-product Lean theorem	Generic real-log/product/sign implication and one concrete five-factor product identity	SxPID event extraction, lattice binding, producer/verifier refinement, or resource-preflight behavior
Verus proof	Contracts for opted-in supported functions	Compiler, external assumptions, or transcendental accuracy
Kani harness	Exhaustive bounded property	Unbounded inputs, concurrent behavior, or numerical accuracy
Exact-count SxPID2 certificate	Conditional outward enclosure for each tool-encoded exact-real expression	Producer correctness, native/compiler correctness, `pid-core` binary64, sampling assumptions, or application validity
Independent Python verifier	Conditional independent SxPID2 event/expression reconstruction and rational-log containment for an accepted exact-count certificate	Formal verification of Python or the verifier, `pid-core` binary64 refinement, sampling assumptions, calibration, or application validity
Exact-product comparison	Conditional exact zero or strict sign for one admitted frozen empirical coordinate	Nonzero magnitude, coordinates whose product preflight abstained, sampling or population sign, or executable refinement
Rocq certificate	Kernel-checked encoded theorem	Correct generator, runtime use, or intended data identity
Hidden benchmark	Observed behavior on a frozen corpus	Universal correctness or a mathematical theorem

Evidence can compose only when a checked bridge binds source revision, tool revision, specification revision, input identity, generated-artifact identity, result identity, assumptions, and claim scope.

10.2 Required evidence envelope

No single value certificate is a complete release-evidence envelope. An accepted qualification bundle must combine the value certificate with records of:

1. exact tool version or commit;
2. compiler, solver, and backend pins;
3. host, target, and installation asset digest;
4. source commit and dirty-state policy;
5. command line;
6. input bounds or theorem statement;
7. assumption inventory;
8. generated artifact hashes;
9. checker result;
10. negative-control result;
11. unsupported cases; and
12. exact permitted claim text.

The present certificate deliberately reports effective dependency-feature resolution, compiled native versions, native archives, and executable identity as unbound or absent. Those absences are compatible with the narrow source-only value claim; they are not compatible with a binary provenance or reproducibility claim. A green exit code without the applicable envelope is development evidence. It is not end-to-end release evidence.

10.3 Fail-closed rules

The certifier must reject invalid or noncanonical input, resource-limit violations, nonpositive log arguments, violated directed-rounding order checks, non-finite authoritative endpoints, malformed or reversed intervals, an empty successive intersection, target-width failure, digest-construction failure, and precision-policy exhaustion. An interval containing zero is not an error: it is a successful outward enclosure with `unresolved_sign`. Future sign- or ordering-specific consumers must fail closed rather than reinterpret that status as positive, negative, zero, or an ordering.

The independent verifier must additionally reject a schema, claim, lattice, expression, interval, source/dependency binding, or payload mismatch, and must reject inability to prove containment. An exact-product preflight abstention is a successful typed absence of zero/sign evidence, not a reason to relabel the interval or infer a sign. If a product record claims `compared`, either implementation must reject inconsistent clearing, projected-resource evidence, product decision, or witness fields. Other layers must still return a typed failure if a required pin or digest differs, an unwinding assertion fails, an assumption is not approved, a certificate has an admission or unexpected axiom, or a supported-subset translator rejects the source. No layer may change a failure into an approximate success.

11 Implemented-lane status and pilot exit criteria

Rug and MPFR. The implemented source lane has exact counts and rational expressions, explicit directed endpoint operations, adaptive precision with typed exhaustion, exact dyadic output, unresolved sign status, resource bounds, source/lock/schema digests, analytic and metamorphic tests, a 494-table numerical corpus, static mutation controls, stable/MSRV CI routes, and a source-only LGPL boundary. This satisfies the narrow reference-tool entry criterion.

Independent verifier. The implemented Python route independently reconstructs the complete averaged SxPID2 event, expression, and lattice semantics and proves rational-log enclosures contained in accepted certificate intervals. Its qualification harness challenges semantic mutations, malformed and resource-amplifying inputs, artifact-binding mismatches, optimizer mode, and hash-seed variation. This is independent executable evidence, not independent formal verification. Stronger claims require deductive verification or a small-kernel proof of the verifier obligations, an external native/build/executable evidence envelope where binary provenance matters, and a proved bridge to any `pid-core` result. Binary distribution still requires a separate license review and relinking/source route.

Exact-product zero/sign extension. The implemented producer and independent verifier separately clear the empirical denominator and compare the admitted exact rational product. Qualification covers all 11,856 coordinates in the 494-table domain, a 308,856-coordinate boundary exhaustion through total count eight, the retained nonempty product-one counterexample, and 23 exact-product-specific adversaries. Resource preflights can abstain without weakening the interval enclosure. This closes exact zero/sign only for a `compared` record; it does not provide a nonzero magnitude, a statistical sign, or a universal theorem about SxPID atoms.

Lean exact-product theorem. Seven theorems under Lean 4.32.0 and pinned mathlib kernel-check the finite log/product identity, its zero and strict-sign consequences, a two-log cancellation example, and the retained five-term product-one identity. Their recorded boundary is generic log/product/sign algebra. Concrete SxPID event extraction, lattice binding, executable refinement, resource accounting, and all sampling or scientific claims remain separate obligations.

Kani. Every harness must declare bounds. Every loop must have a passing unwinding assertion. Meaningful mutations must produce counterexamples. No harness can model transcendental accuracy. The report must state the bounded domain.

Verus. The complete selected kernel must verify. The assumption inventory must be empty or approved. Semantic mutations must fail. The erased program must pass Rust 1.89 gates. Ordinary tests must agree with specification fixtures.

Rocq Interval. The schema must be versioned. `rocqcheck-o` must report no unexpected assumption. The gate must reject admissions. Identity mutations must fail. Archived exact inputs must regenerate the certificate.

Aeneas. The selected subset must translate without an unsound workaround. External models must be minimal and reviewed. The theorem correspondence must be documented. Lean compatibility must be resolved. Source mutations must affect the model and proof.

12 Conclusion

The categorical interval producer and an independent exact-integer/Fraction rational-log verifier have been implemented for the complete averaged two-source SxPID lattice. The verifier closes the earlier executable semantic-cross-check gap by reconstructing event masses, exact expressions, lattice coordinates, dyadic endpoints, and artifact bindings without trusting Rug/MPFR conclusions. The separately bounded exact-product extension adds complete zero/strict-sign decisions when its preflight admits a coordinate, and a Lean module checks only the corresponding generic algebra and one retained product-one identity. These results are complementary: the interval encloses magnitude, the exact product decides zero/sign, and Lean checks an abstract mathematical implication. None formally verifies Python or connects either executable to `pid-core`.

The highest-value next step is bounded bit-precise evidence: Kani should challenge compiled integer, event, mask, lattice, overflow, and rejection behavior under explicit bounds and unwinding assertions. After those bounded checks expose and stabilize the executable kernel, Verus or Creusot can target unbounded functional contracts for the same small pure kernel.

Rocq Interval remains useful as an optional small-kernel proof route, provided its theorem statement is not copied unquestioningly from either executable implementation. Aeneas remains a research route whose release and Lean-version state does not justify production adoption.

> `pid-rs` is not end-to-end formally verified. The source-only certifier emits conditional enclosures, and the independent executable verifier reconstructs the averaged categorical SxPID2 semantics and proves containment subject to its Python and mathematical TCB. A separately bounded rational-product lane conditionally proves exact zero or strict sign, and the Lean theorem checks only its generic algebraic core. Every stronger result must state the exact layer, expression or theorem, assumptions, tool pins, trust boundary, build and input bindings, and unsupported claims.

13 Primary source index

Verus

- Release `0.2026.07.18.3a4d30b`
- Verus guide
- Cargo integration
- Trusted computing base
- Assumption specifications

Kani

- Release 0.67.0
- Installation guide
- Kani manual
- Loop unwinding
- Rust feature support
- Intrinsic support

Aeneas and Charon

- Aeneas repository
- Aeneas releases
- Pinned Aeneas commit

MPFR and Rug

- MPFR project
- MPFR 4.2.2
- MPFR manual
- Rug 1.30.0
- `gmp-mpfr-sys` 1.7.1
- GMP copying conditions

Rocq and Interval

- Rocq project
- Rocq 9.2.0
- `coq-interval` 4.11.4
- Interval documentation
- Rocq OPAM instructions
- Rocq compile and check tools